A Layered Authorization Model for Scalable Enterprise Systems

Balancing Centralized Identity and Application-Level RBAC in Cloud Architectures

Contents

Abstract 2

1. Introduction 2

2. Problem Statement 2

3. Proposed Architecture 3

   3.1 Authentication Layer 3

   3.2 Platform-Level Authorization Layer 3

   3.3 Internal Authorization Service (Private API) 3

   3.4 Application-Level Authorization (RBAC) 3

4. Architecture Diagram 4

5. Trade-Off Analysis 4

   5.1 Centralization vs Flexibility 4

   5.2 Control vs Scalability 5

6. Key Design Principles 5

   6.1 Separation of Concerns 5

   6.2 Layered Decision-Making 5

   6.3 Avoid Over-Centralization 5

   6.4 Design for Evolution 5

7. Discussion 5

8. Conclusion 5

Abstract

Modern enterprise platforms increasingly support multiple applications within a shared cloud environment, introducing significant challenges in managing identity, authorization, and access control. Traditional approaches tend to favor either centralized control, which improves consistency but reduces flexibility, or decentralized models, which allow application-level autonomy but introduce governance and scalability issues.

This paper proposes a layered authorization model that separates authentication, platform-level authorization, and application-level role-based access control (RBAC). The model leverages cloud-native services such as AWS Cognito and API Gateway, along with internal authorization services, to balance consistency, flexibility, and scalability. Through architectural analysis and practical considerations, this paper outlines key design principles and trade-offs relevant to enterprise system design.

1. Introduction

As enterprise systems evolve toward platform-based architectures that support multiple applications and user groups, the complexity of managing identity and access control increases significantly.

A fundamental design question arises:

Should authorization be centrally managed across the platform, or delegated to individual applications?

Fully centralized models ensure consistency and governance but often fail to support diverse business requirements. Conversely, fully decentralized models provide flexibility but lead to fragmented identity management and increased operational complexity.

This paper presents a layered approach to authorization that distributes decision-making across system layers while maintaining a unified identity model.

2. Problem Statement

In multi-application enterprise environments, access control must satisfy several competing requirements:

- **Consistency**: A unified identity and access model across applications
- **Flexibility**: Application-specific permissions and business logic
- **Scalability**: Ability to support growing users and services
- **Governance**: Central oversight and auditability

These requirements create inherent tension:

- Centralization improves governance but limits adaptability
- Decentralization improves flexibility but reduces control

The challenge is not choosing one approach, but designing a system that balances both.

3. Proposed Architecture

The proposed model introduces a **layered authorization architecture**, separating concerns across four distinct layers:

3.1 Authentication Layer

Authentication is handled by a centralized identity provider (e.g., AWS Cognito), responsible for:

- User authentication
- Token issuance (JWT)
- Identity validation

This layer establishes a single source of truth for user identity.

3.2 Platform-Level Authorization Layer

An API Gateway-based authorizer validates incoming requests:

- Verifies JWT tokens
- Enforces platform-level access rules
- Controls access to shared services

This layer determines whether a user is allowed to interact with the platform.

3.3 Internal Authorization Service (Private API)

A private, internal API manages:

- User-role relationships
- Application-specific permissions
- Organizational structures

This service is not publicly exposed and acts as a centralized data source for authorization decisions.

3.4 Application-Level Authorization (RBAC)

Individual applications implement their own RBAC logic:

- Interpret user roles and context
- Apply business-specific rules
- Determine allowed actions

This enables each application to retain flexibility while relying on shared identity infrastructure.

4. Architecture Diagram

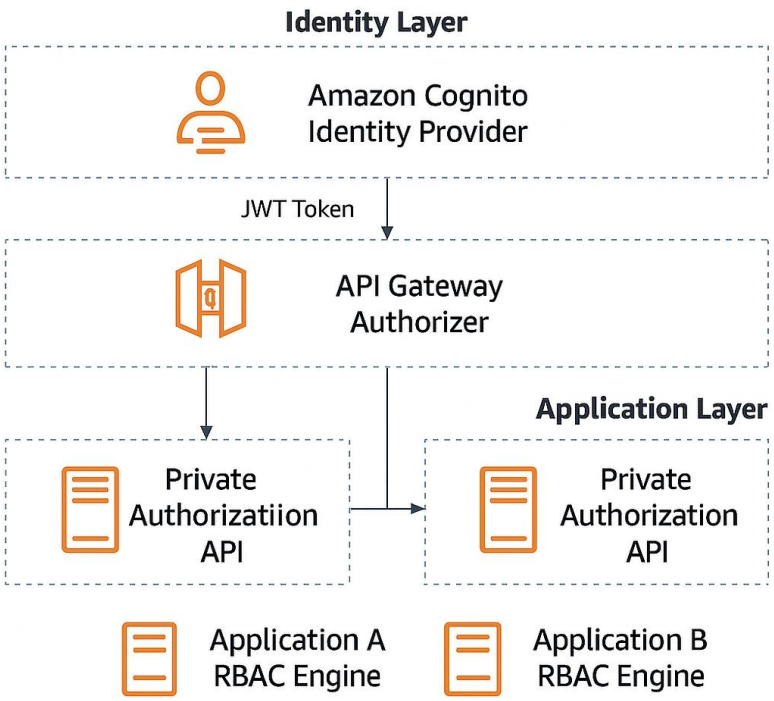

Figure 1. Layered Authorization Architecture for Enterprise Systems

5. Trade-Off Analysis

5.1 Centralization vs Flexibility

Approach	Advantages	Limitations
Centralized	Consistency, governance	Limited flexibility
Decentralized	Flexibility, customization	Fragmentation
Layered (Proposed)	Balanced	Increased architectural complexity

Table 1. Trade-off Analysis of Authorization Models

5.2 Control vs Scalability

- Central control improves security and auditability
- Distributed decision-making improves scalability

The layered model enables both by separating concerns.

6. Key Design Principles

6.1 Separation of Concerns

Authentication, authorization, and business logic must be decoupled to maintain system clarity and scalability.

6.2 Layered Decision-Making

Authorization decisions should be made at the appropriate level:

- Platform → access to the system
- Application → allowed actions

6.3 Avoid Over-Centralization

Not all authorization logic belongs at the platform level. Over-centralization reduces adaptability.

6.4 Design for Evolution

Enterprise systems must adapt to changing requirements. Authorization models should support incremental evolution without requiring system-wide redesign.

7. Discussion

The proposed model demonstrates that effective authorization design is not about choosing between centralized and decentralized models, but about structuring how decisions are distributed.

By centralizing identity and selectively decentralizing authorization, the system achieves both governance and flexibility. This approach aligns with modern cloud-native and microservice-based architectures, where autonomy and coordination must coexist.

8. Conclusion

This paper presents a layered authorization model for enterprise systems that balances centralized identity management with application-level flexibility.

The key insight is that:

Well-designed systems do not centralize all decisions, but place decisions at the appropriate level of abstraction.

This approach enables scalable, maintainable, and adaptable enterprise platforms capable of supporting diverse applications and evolving requirements.